RAJALAKSHMI ENGINEERING COLLEGE

Padmapriya S K 251801218-2025-AIDS

P2

Started on	Saturday, 5 September 2026, 11:35 AM
State	Finished
Completed on	Saturday, 5 September 2026, 11:36 AM
Time taken	34 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findPair(int arr[], int left, int right, int x) {

    // Base case
    if (left >= right) {
        return 0;
    }

    int sum = arr[left] + arr[right];

    // Pair found
    if (sum == x) {
        printf("%d\n", arr[left]);
        printf("%d\n", arr[right]);
        return 1;
    }

    // If sum is greater than x, move right pointer
    if (sum > x) {
        return findPair(arr, left, right - 1, x);
    }

    // If sum is less than x, move left pointer
    return findPair(arr, left + 1, right, x);
}

int main() {

    int n, x;

    // Read size of array
    scanf("%d", &n);

    int arr[n];

    // Read array elements
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Read target sum
    scanf("%d", &x);

    // Check for pair
    int found = findPair(arr, 0, n - 1, x);

    if (found == 0) {
        printf("No\n");
    }
```

```
51
52      return 0;
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course